

Automatic Alert System Using Joint Architecture for NER and SA.

Rodrigo Covas Vasquez, Mario Alonso Alonso, Marcos Garrido Ferrer

April 2025

1 Introduction

The main objective of this work is to develop a model capable of detecting named entities in a sentence and assigning a sentiment (positive, neutral, or negative) to it.

First, a database will be created by combining real and synthetic elements, containing sentences along with their corresponding entities and sentiments, which will be used to train and validate the model. Next, the model will be obtained and used to generate alerts through another pre-trained model. A file containing sentences will be received, and these will be processed by our model to identify the relevant entities and sentiments, in order to inform the pre-trained model on how to construct the alert.

The interest of this work lies in the lack of well-known models that simultaneously include both NER (Named Entity Recognition) and SA (Sentiment Analysis).

The code of the project is written in Python and is based on the PyTorch library for tensor calculations. The source code can be found in the following GitHub Repository.

2 Dataset

We did not find any publicly available datasets featuring both Named Entity Recognition and Sentiment Analysis. Our first approach was to use a pre-trained Sentiment Analysis model to assign sentiment tags to a NER dataset's phrases. However, we quickly found this approach ineffective, as our model's SA predictions were not accurate enough (roughly surpassing a random classification model's accuracy). Thus, we settled on employing two separate datasets: the CoNLL2003 dataset for NER and the Financial Phrasebank dataset for SA. We tokenized both datasets' phrases and translated them to BERT embeddings that we could feed to our model.

2.1 CoNLL2003 dataset

This dataset contains tokenized phrases and an individual NER tag for each of the tokens. These tags can be of nine different types:

- O: Not part of any named entity
- B-PER, I-PER: Beginning and inside of a *person* name
- B-ORG, I-ORG: Beginning and inside of an *organization* name
- B-LOC, I-LOC: Beginning and inside of a *location* name
- B-MISC, I-MISC: Beginning and inside of a *miscellaneous* named entity (e.g., nationalities, events, etc.)

Notably, as typically most of the tokens in any given phrase are not part of a named entity, the vast majority of the NER tags in the dataset are of index 0. When we trained our first models, we observed that their accuracy skyrocketed in a matter of 5 epochs; reaching upwards of 90%. However, this was due to the fact that these models were assigning the "Not a part of any named entity" tag to almost every single token. In light of this, we decided to take into account the frequency of each tag. Incorrectly classifying tags that appear fewer times (such as the "Beginning of an organization name") entails a higher loss. Specifically, given the count of each class, the weight for class i is calculated as:

$$w_i = \frac{1}{c_i + \epsilon}$$

where c_i is the count of class i , and $\epsilon = 10^{-6}$ is a small constant to prevent division by zero. The weights are then normalized to ensure they sum to the number of classes:

$$w_i = w_i \cdot \frac{N}{\sum_j w_j}$$

where N is the total number of classes.

The loss formula for each token is calculated as:

$$\text{loss} = \mathbf{1}_{\{y \neq \hat{y}\}} \cdot w_i$$

where y is the true label, $\hat{y}$ is the predicted label, $\mathbf{1}_{\{y \neq \hat{y}\}}$ is an indicator function that equals 1 when the prediction is incorrect and 0 otherwise, and w_i is the pre-computed weight for class i .

2.2 Financial Phrasebank dataset

The Financial Phrasebank Dataset contains sentences from financial news, each of them with their respective sentiment label. There are four versions of this dataset, according to the number of annotators that agreed with the labels:

We decided to employ `sentences_66agree`, as we deem it has the optimal balance between reliability and size; the latter being pivotal at the training stage of the model.

Agreement Level	Number of Instances
<code>sentences_50agree</code> ($\geq 50\%$)	4846
<code>sentences_66agree</code> ($\geq 66\%$)	4217
<code>sentences_75agree</code> ($\geq 75\%$)	3453
<code>sentences_allagree</code> (100%)	2264

Table 1: Number of sentences with different levels of annotator agreement

3 Model

Initially, our model contained two distinct LSTMs: one for NER and one for SA. However, our current model is a joint architecture designed to perform NER and SA simultaneously.

3.1 Architecture

The core of our model is a multi-layer LSTM network. It receives input embeddings of size $(\text{batch_size}, \text{seq_len}, \text{input_dim})$ and outputs hidden states $\mathbf{H} \in \mathbf{R}^{\text{batch_size} \times \text{seq_len} \times (\text{hidden_dim} \times d)}$, where $d = 2$ for bidirectional LSTMs and $d = 1$ otherwise.

Named Entity Recognition: For the NER task, the model applies a linear classification layer to each token in the output sequence $\mathbf{H}$, producing token-level class scores:

$$\text{NER logits} \in \mathbf{R}^{\text{batch_size} \times \text{seq_len} \times \text{num_ner_classes}}$$

Sentiment Analysis: For sentiment classification, the model extracts the final hidden state(s) from the last LSTM layer. If the LSTM is bidirectional, the final forward and backward hidden states are concatenated to form a sentence-level representation:

$$\mathbf{h} = [\mathbf{h}_{\text{last}}^{\rightarrow}; \mathbf{h}_{\text{last}}^{\leftarrow}] \in \mathbf{R}^{\text{hidden_dim} \times 2}$$

This representation is then passed through a separate linear layer to obtain sentiment class scores:

$$\text{SA logits} \in \mathbf{R}^{\text{batch_size} \times \text{sentiment_classes}}$$

3.2 Training stage

We train our model sequentially due to its architecture. First, we train the model using only the CoNLL2003 dataset (embeddings and NER tags). At this stage, only the NER predictions of the model are taken into account when calculating the loss. Thus, the model is only trained to classify NER tags correctly. As previously explained (see the Dataset section for more details), the loss entailed

by a wrong classification is inversely proportional to the frequency of the target label in the original dataset, to account for class imbalance.

We reduce the learning rate by a factor of 0.5 when the model does not improve 2 epochs in a row. Additionally, we use an early stopping mechanism that triggers when the model’s loss is not lowered by at least 0.001 for 10 consecutive epochs. Whenever we train a model that has lower validation loss than our current one, we discard the latter and save our new model in memory.

Afterwards, we freeze the encoder parameters using `param.requires_grad = False`. This ensures that what was learnt during the previous stage cannot be modified. We now proceed as before, only now the model’s SA predictions of the Financial Phrasebank dataset are taken into account.

3.3 Evaluation

After training our final model, we obtained the following accuracy results:

Task	Train Accuracy	Validation Accuracy
NER	0.967	0.911
SA	0.614	0.682

Table 2: Training and validation accuracy for NER and Sentiment Analysis tasks after final model training.

Notably, the SA validation accuracy is 0.682, which seems reasonable as we selected the version of the Financial Phrasebank dataset of labels with which 66% of respondents agree.

3.4 Conclusions

We have developed a multi-task learning architecture that performs NER and SA simultaneously. We trained it using two different publicly available datasets: one for each task.

4 Alert generation

To perform alert generation, several pre-trained models were tested. After discarding those that require access keys and those that take up several gigabytes of storage, the remaining ones produced rather unsatisfactory results. Ultimately, the chosen strategy was to use a text generator based on prompts, in which the model is informed of the sentence from which it must generate the alert, the entities it must mention, and the sentiment it should convey in the response.

The selected model was “google/flan-t5-base”, which, although not highly accurate, was the closest to what was needed. Tests were also conducted with models like “facebook/bart-base” and “gpt2”, among others.

The execution is simple: in a ‘.txt’ file located in the “data” folder, the sentences from which alerts are to be generated are stored, and by simply running the ‘alerts.py’ file, the sentences, recognized entities, detected sentiment, and the generated alert will be printed sequentially.

The operation of “alerts.py” is based, first of all, on generating embeddings from the input sentences, just as was done to build the initial dataset. By passing these embeddings through the previously trained model, the entities and sentiments are obtained. With the original sentences, their detected entities, and the detected sentiments, the generative model has all the information it needs to process and construct the alert.

Here is an example of the alert generation output:

Example: Sentence 14

Sentence: *The British Museum opened a new exhibition on Ancient Egypt.*

Sentiment: Neutral

Entities:

- ('The', '0')
- ('British', '0')
- ('Museum', '0')
- ('opened', '0')
- ('a', '0')
- ('new', '0')
- ('exhibition', '0')
- ('on', '0')
- ('Ancient', '0')
- ('Egypt', 'Localization')
- ('.', '0')

NER-highlighted Sentence:

British Museum opened a new exhibition on **Ancient Egypt**

5 Conclusions

In this project, we have developed and evaluated a multi-task learning architecture capable of performing both Named Entity Recognition (NER) and Sentiment Analysis (SA). We trained our model using two publicly available datasets; one for each of the tasks.

The model architecture at its core is bidirectional LSTM encoder, with two output heads. This design allows for shared feature learning. The results show high accuracy on the NER task, with validation accuracy reaching 91.1%, and a reasonable performance on the SA task, with a validation accuracy of 68.2%.

The discrepancy in performance between the two tasks may be attributed to differences in dataset quality or complexity. Nonetheless, the architecture demonstrates the potential of a unified model for joint sequence labeling and sentence classification tasks in natural language processing.

Our alert generation mechanism, based on a pre-trained model, processes a batch of phrases and returns automatically generated alerts based on the perceived sentiment of each sentence.